Designing Data Structures

to *Uncover*

Pharo-AI *Optimisation Opportunities*

Pharo + Lapack vs scikit-learn

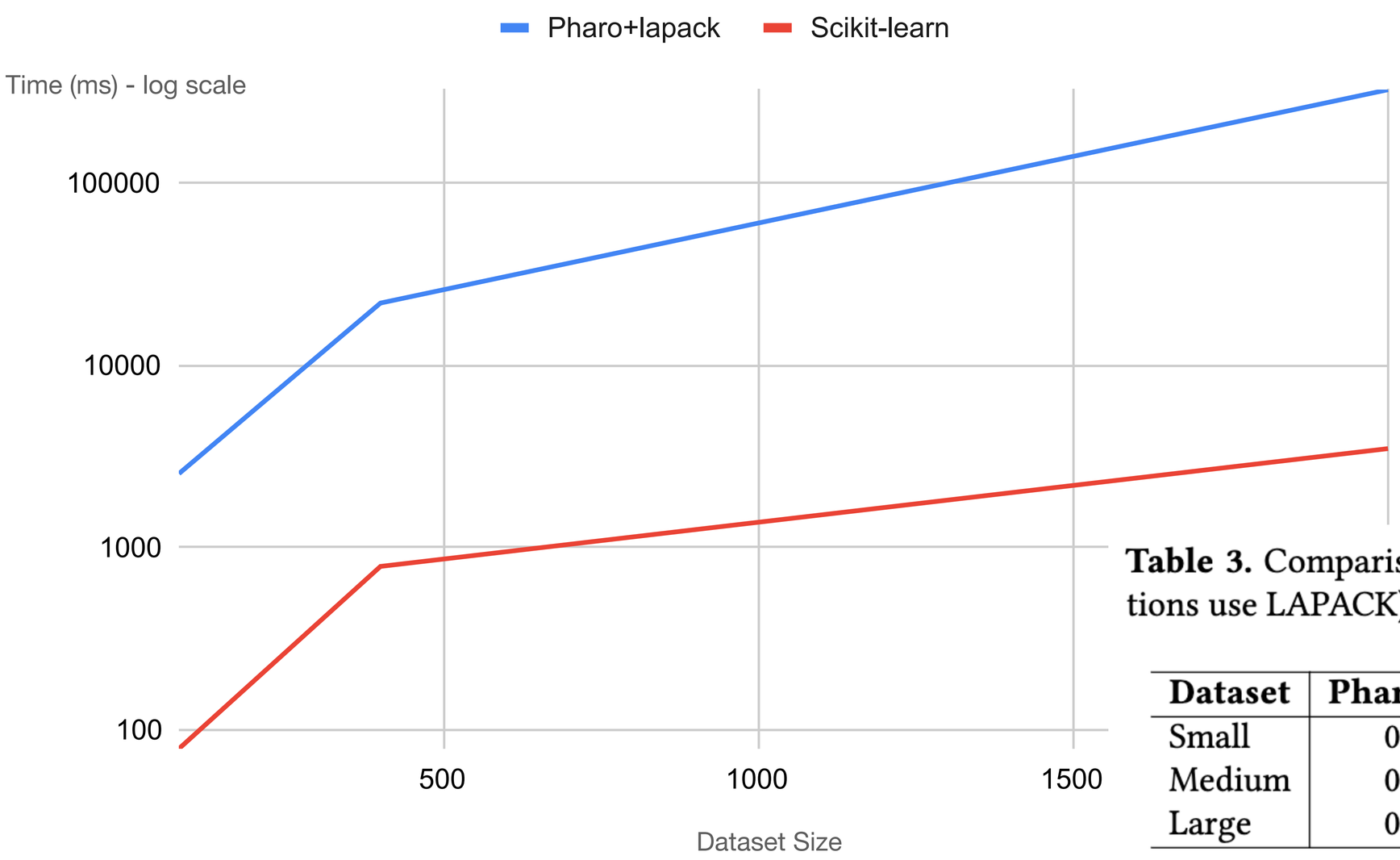
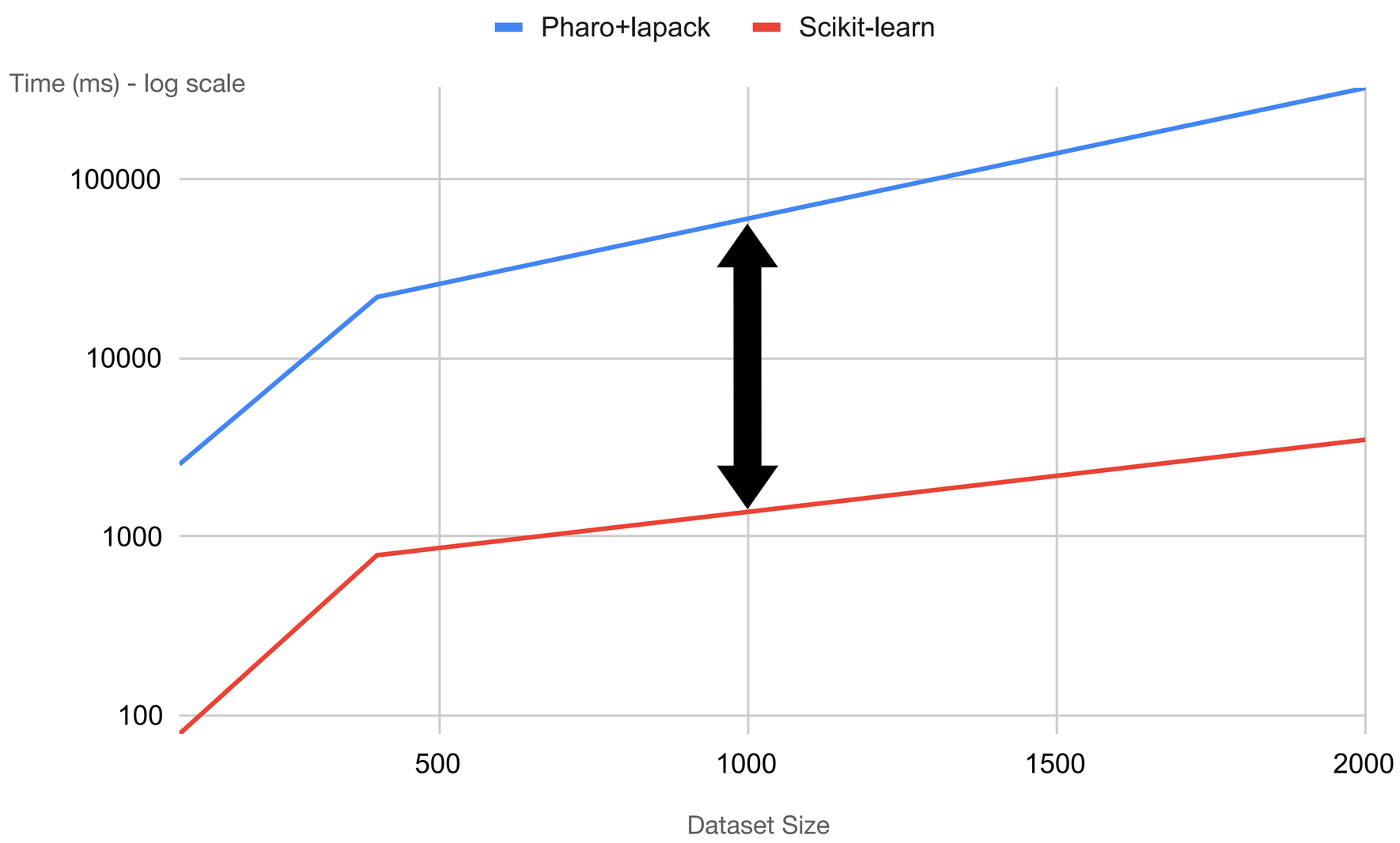


Table 3. Comparison with scikit-learn (both implementations use LAPACK).

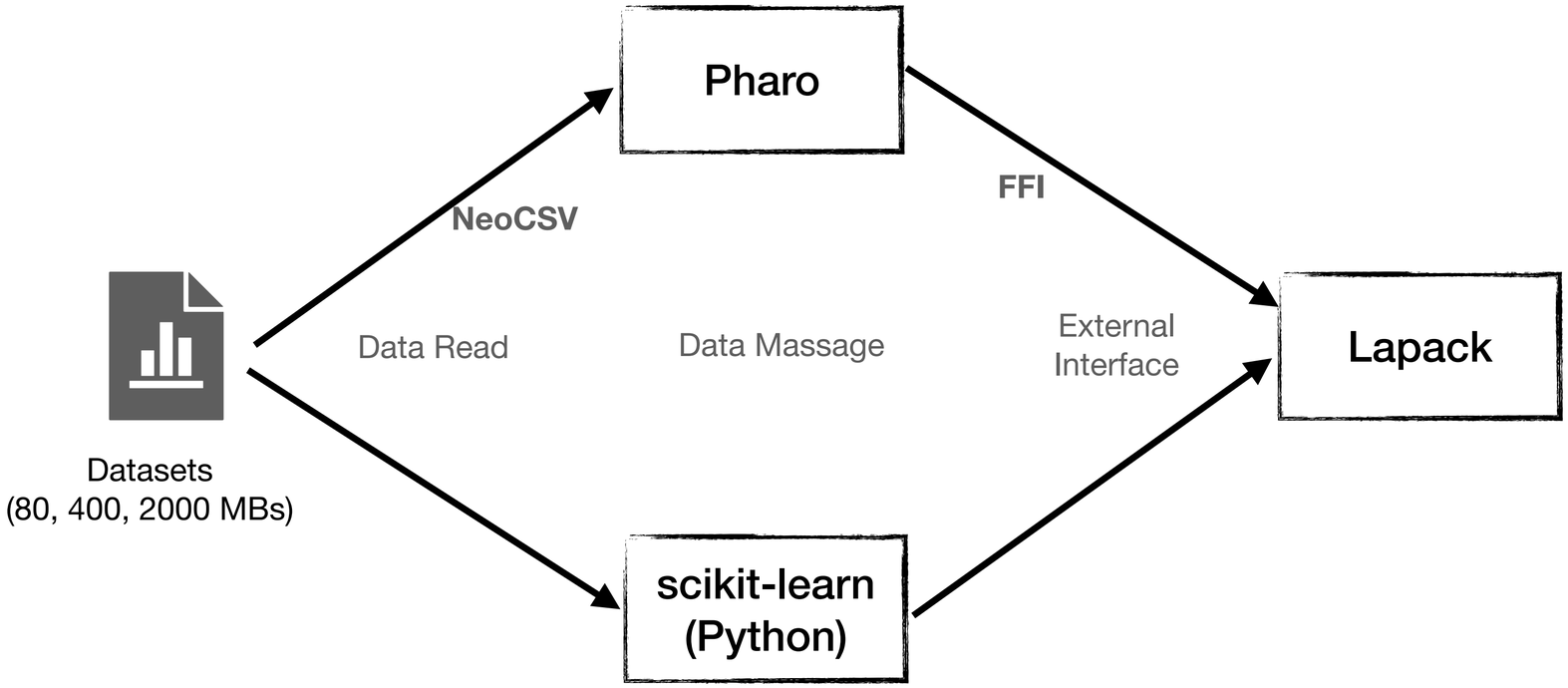
Dataset	Pharo & LAPACK	scikit-learn	Diff
Small	00:00:02.551	00:00:00.079	32×
Medium	00:00:22.000	00:00:00.790	28×
Large	00:05:25.798	00:00:03.499	93×

Quick analysis

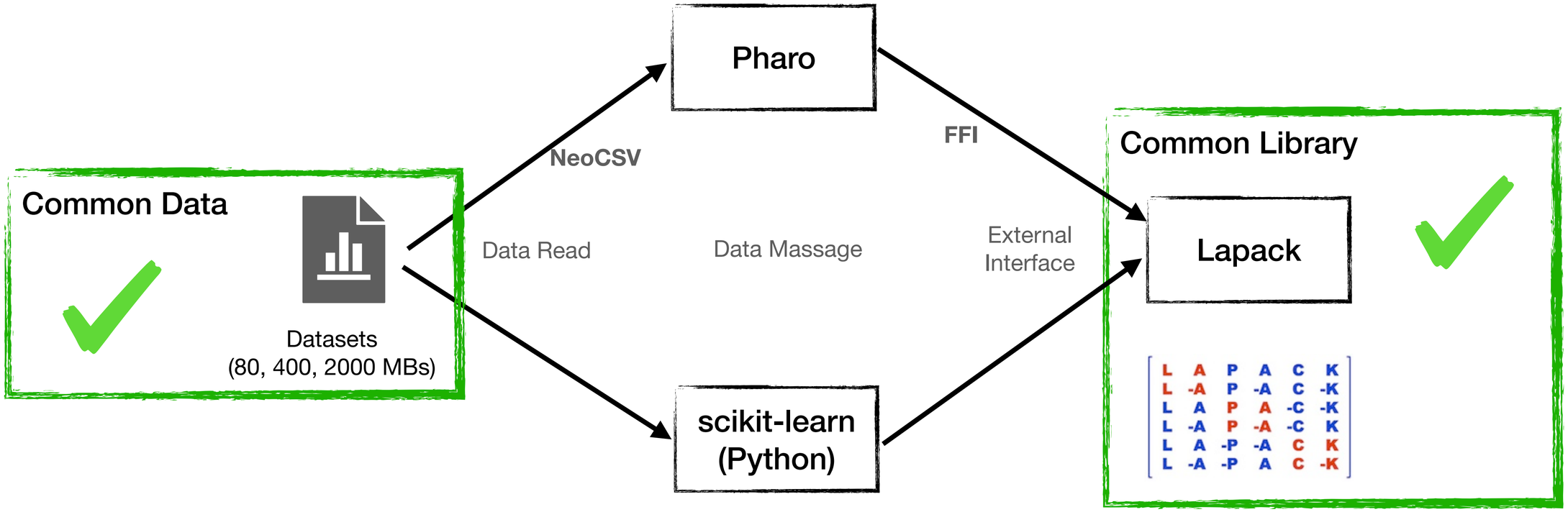


- Similar tendency
- *Seems* between linear and exponential (but only 3 points…)
- Gap *seems ~constant*
  - => Difference keeps same order of magnitude

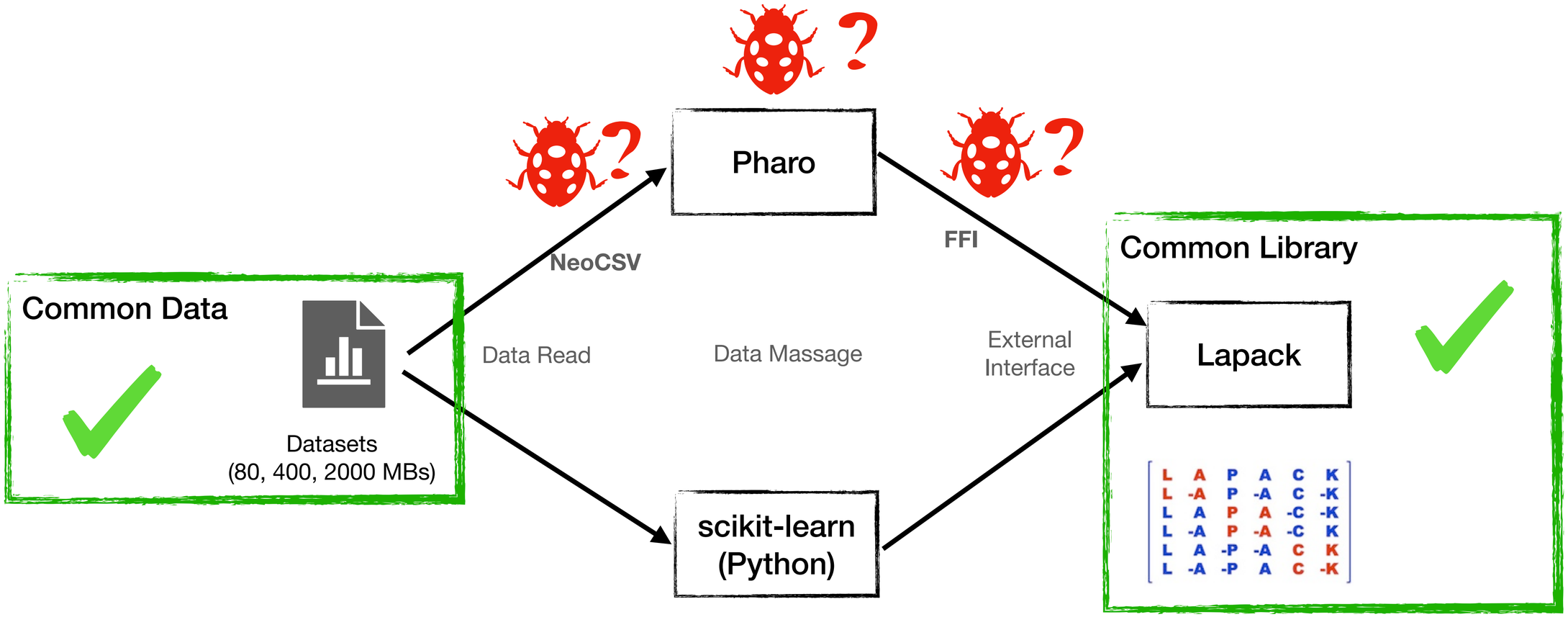
Understanding the Scenario



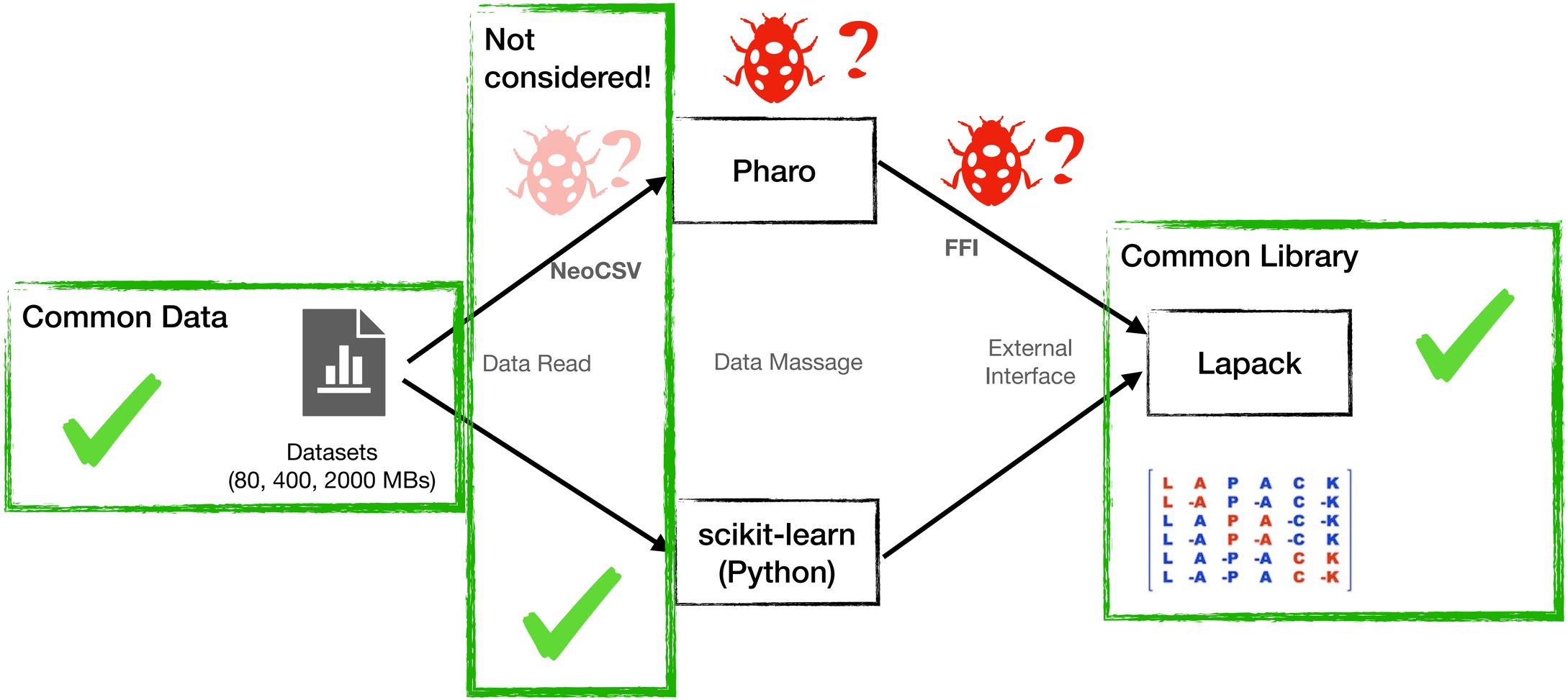
Hypothesizing Performance Problems



Hypothesizing Performance Problems



Hypothesizing Performance Problems



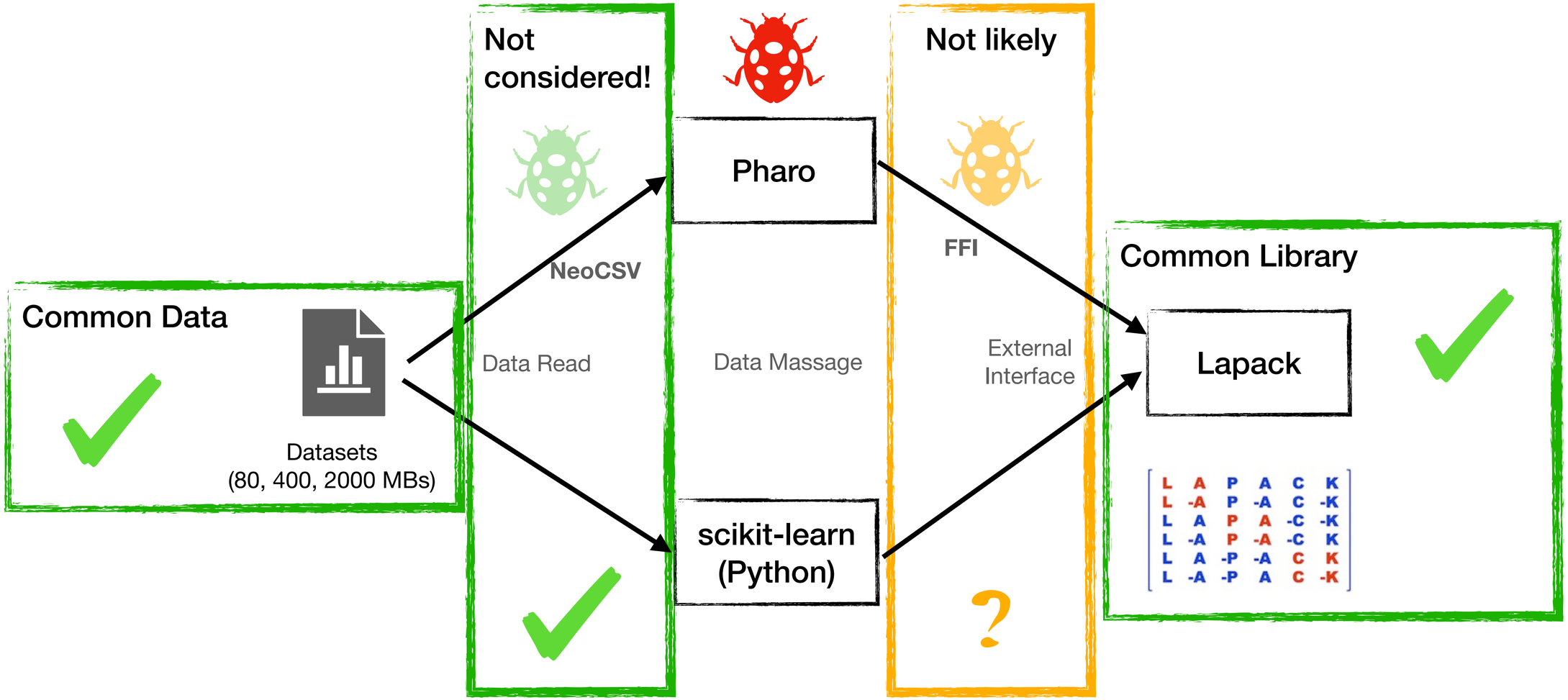
FFI Performance insights

- Pharo FFI can perform **~4 million calls per second**
  (on a x86-64 Haswell 2015 Macbook Pro)

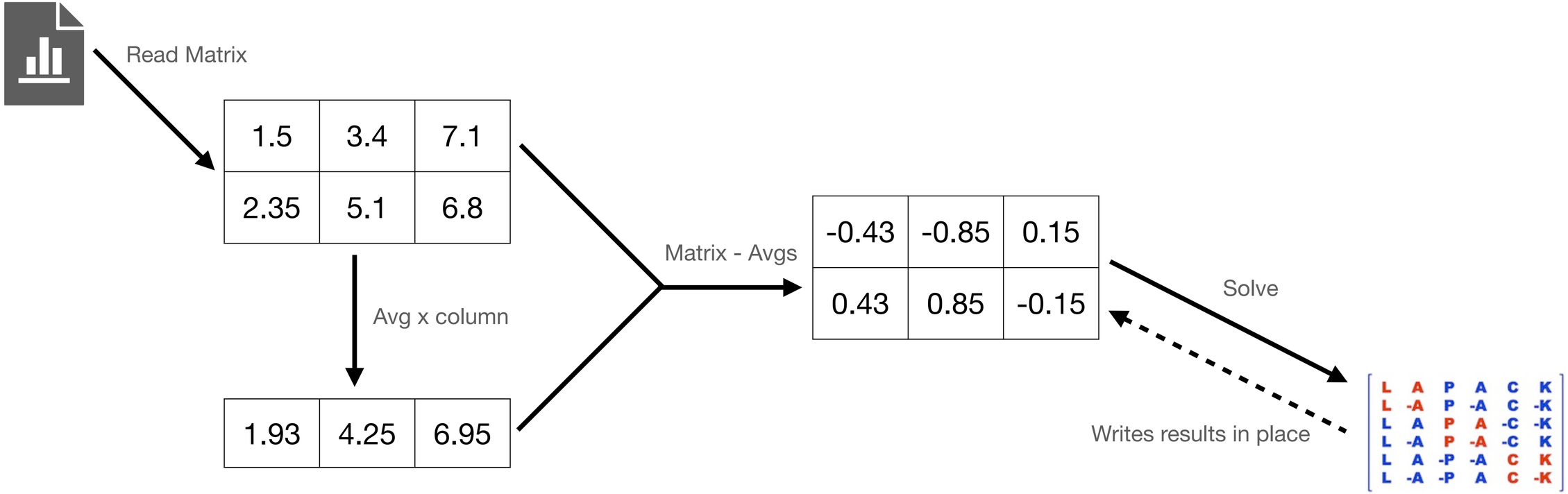
- Each call takes ~0.25 μ seconds (micro seconds)
  => at first sight, this should ***not be a bottleneck***

- Pharo FFI *avoids copying* big arrays/objects if used properly

Hypothesizing Performance Problems

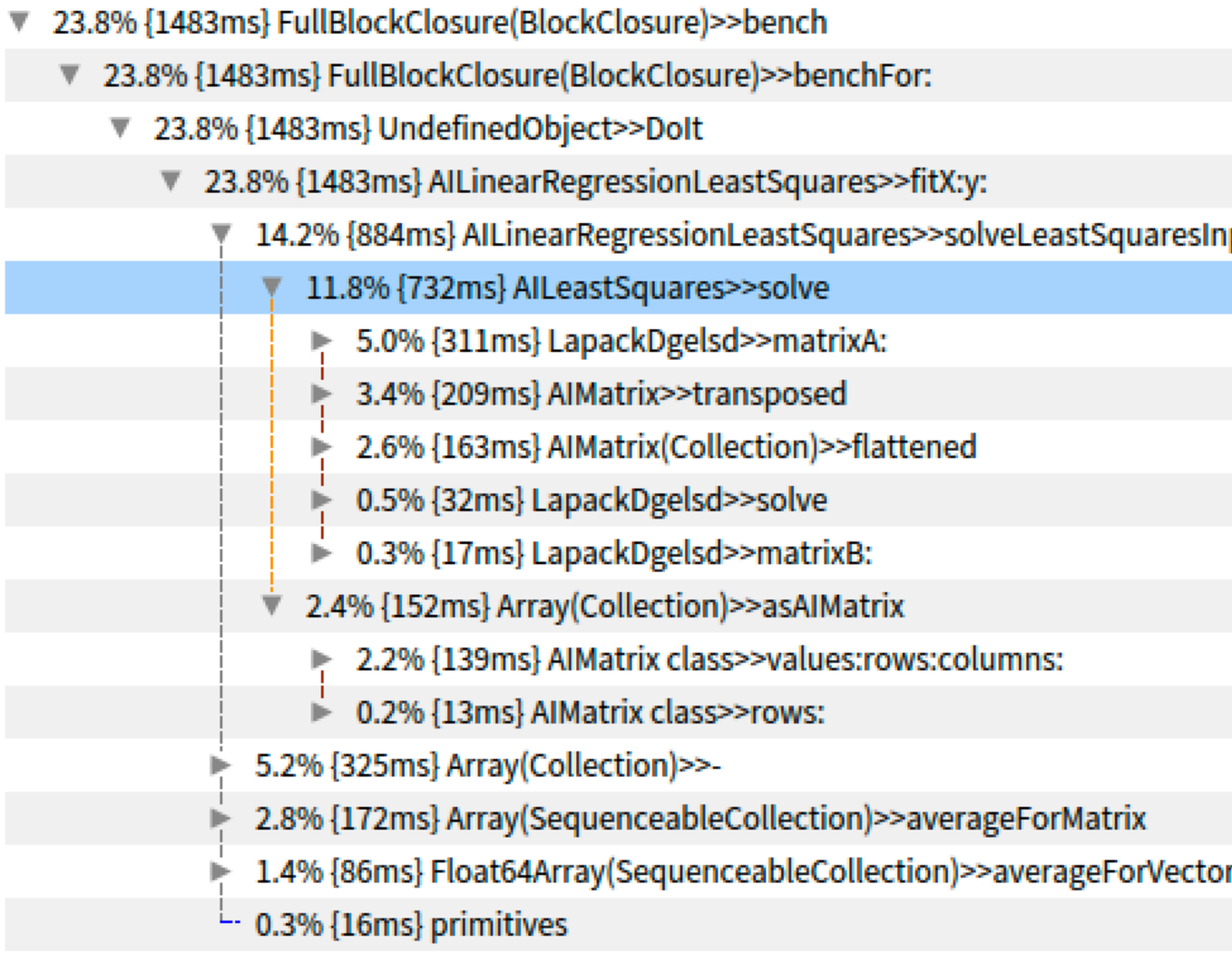


The Linear Regression Implementation

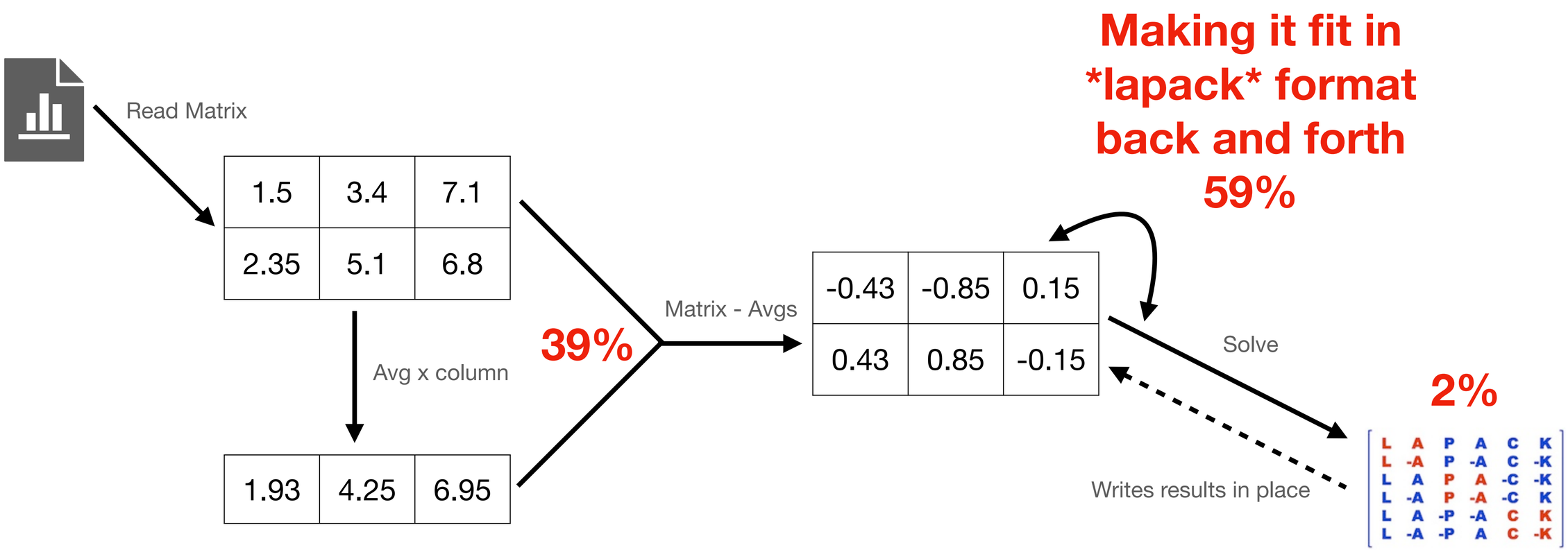


Understanding the Performance Profile

- Lapack call is **~2%**
- Data preparation **~39%**
  - averages
  - subtractions
- Other data movements **~59%**

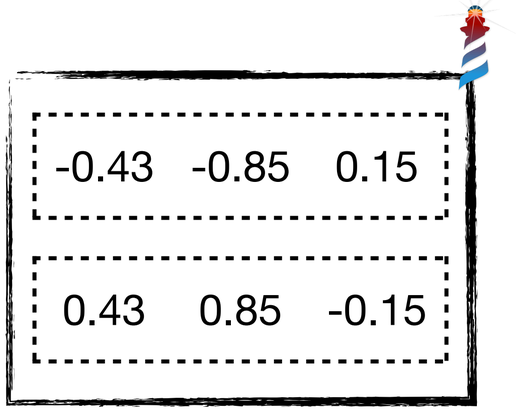


Performance Profile in Perspective

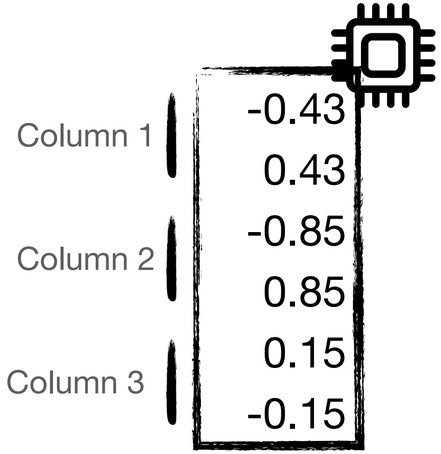


The Hidden 60%

-0.43	-0.85	0.15
0.43	0.85	-0.15



Array of rows with Pharo floats



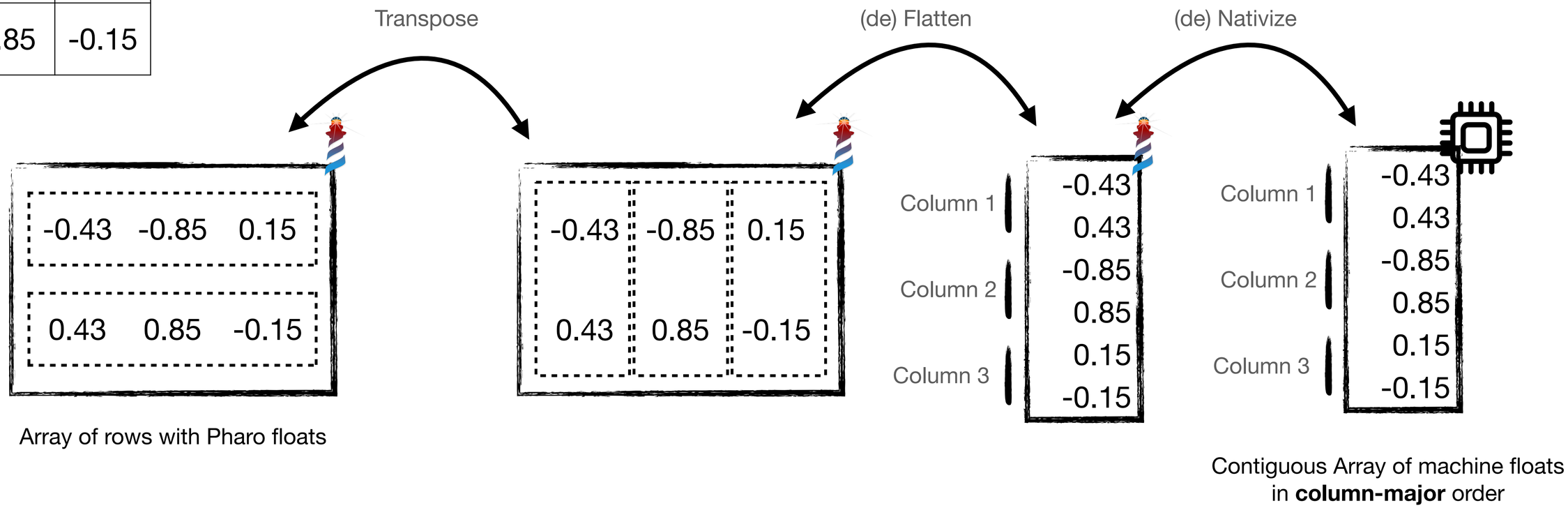
Contiguous Array of machine floats
in **column-major** order

Pharo-AI representation

Lapack's expectations

The Hidden 60%

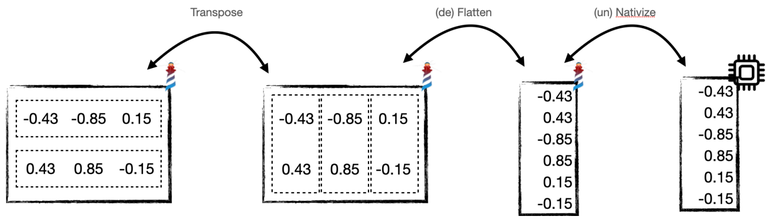
-0.43	-0.85	0.15
0.43	0.85	-0.15



Pharo-AI representation

Lapack's expectations

Understanding Performance Bottlenecks

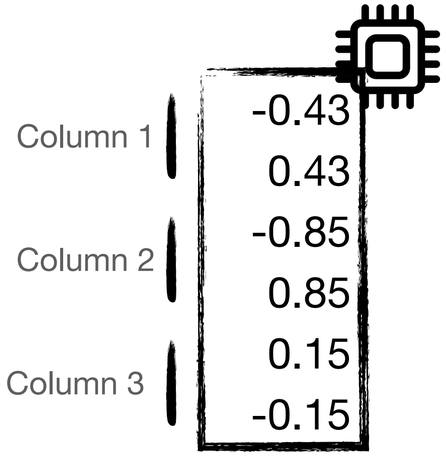


- Data movement is expensive
  - System Caches are small
  - Efficient (software-based) Matrix transposition is still active research!
  - And here we have 3 data copies to go, 3 to come back (!!)

- Insight => Use an internal representation similar to Lapack's
  - Avoid copies, transpositions, float transformations

Proposal 1: Contiguous Array of Native Floats

-0.43	-0.85	0.15
0.43	0.85	-0.15

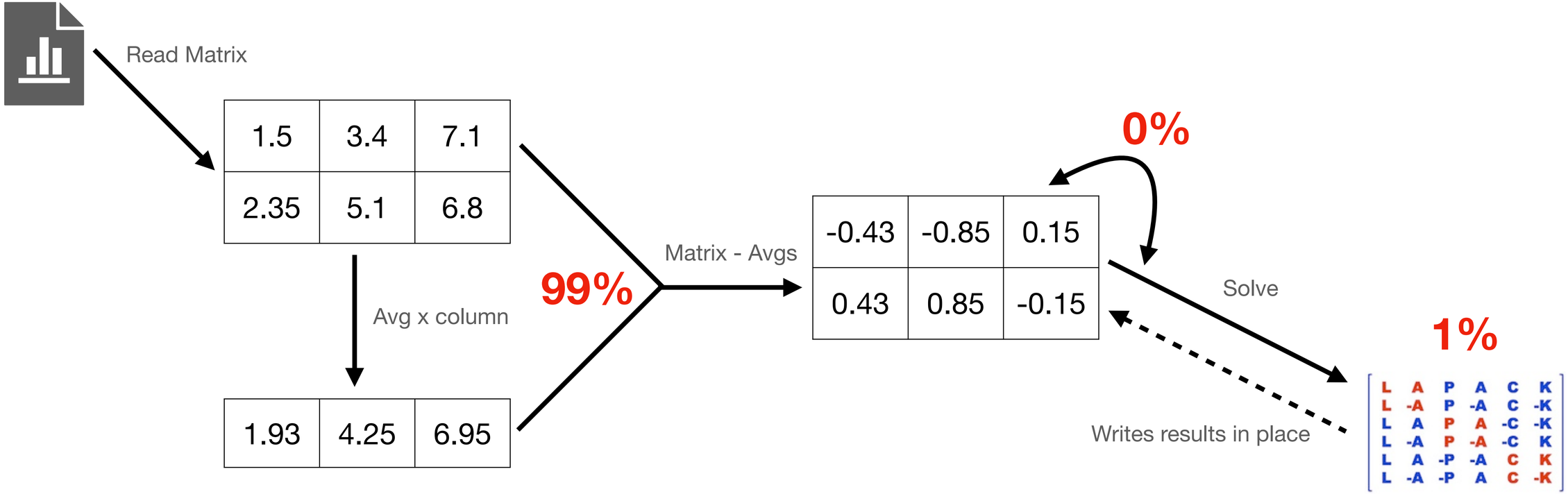


Contiguous Array of machine floats
in **column-major** order

Pharo-AI representation

Lapack's expectations

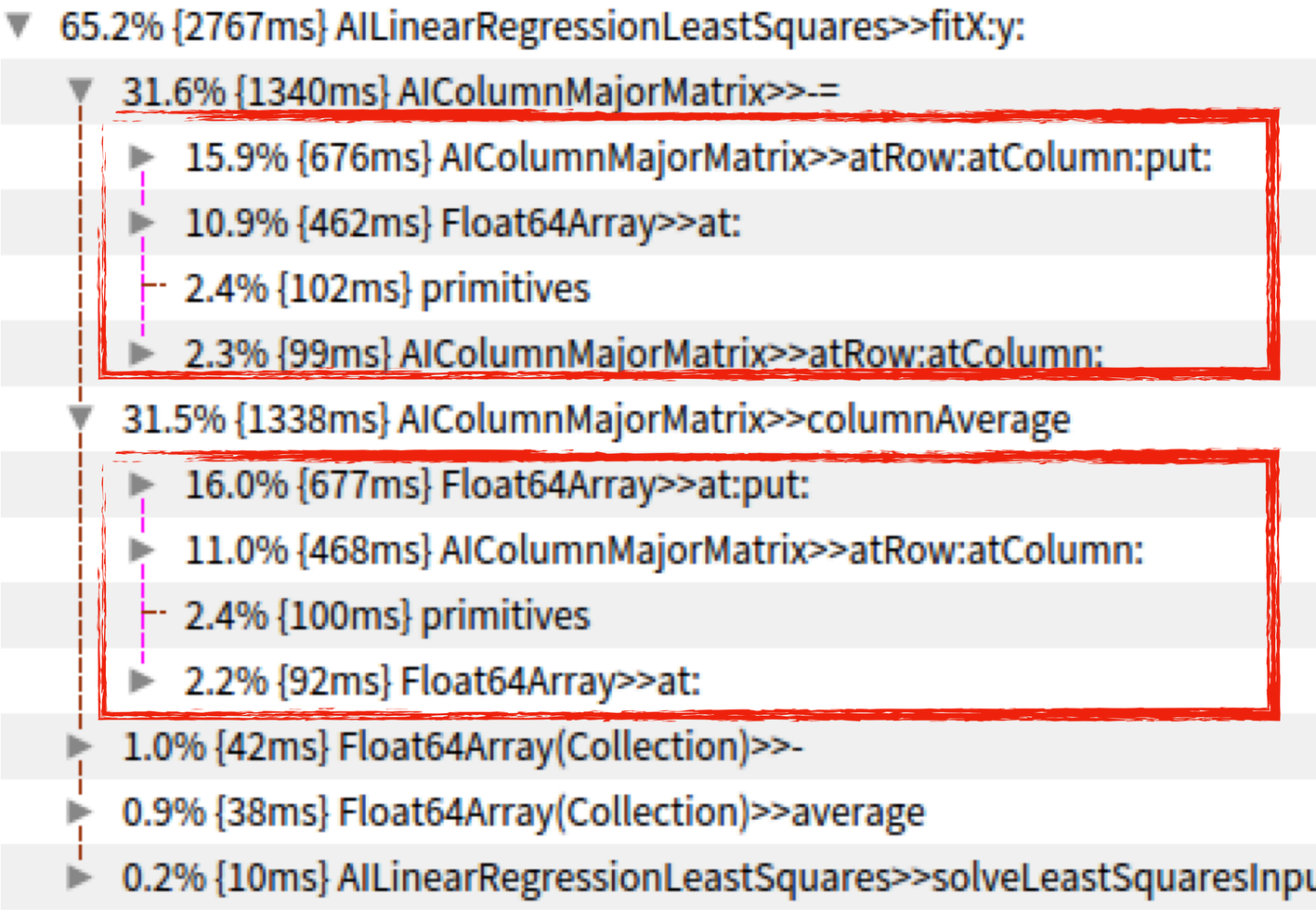
Proposal 1 Performance



But 0.66x slower :(

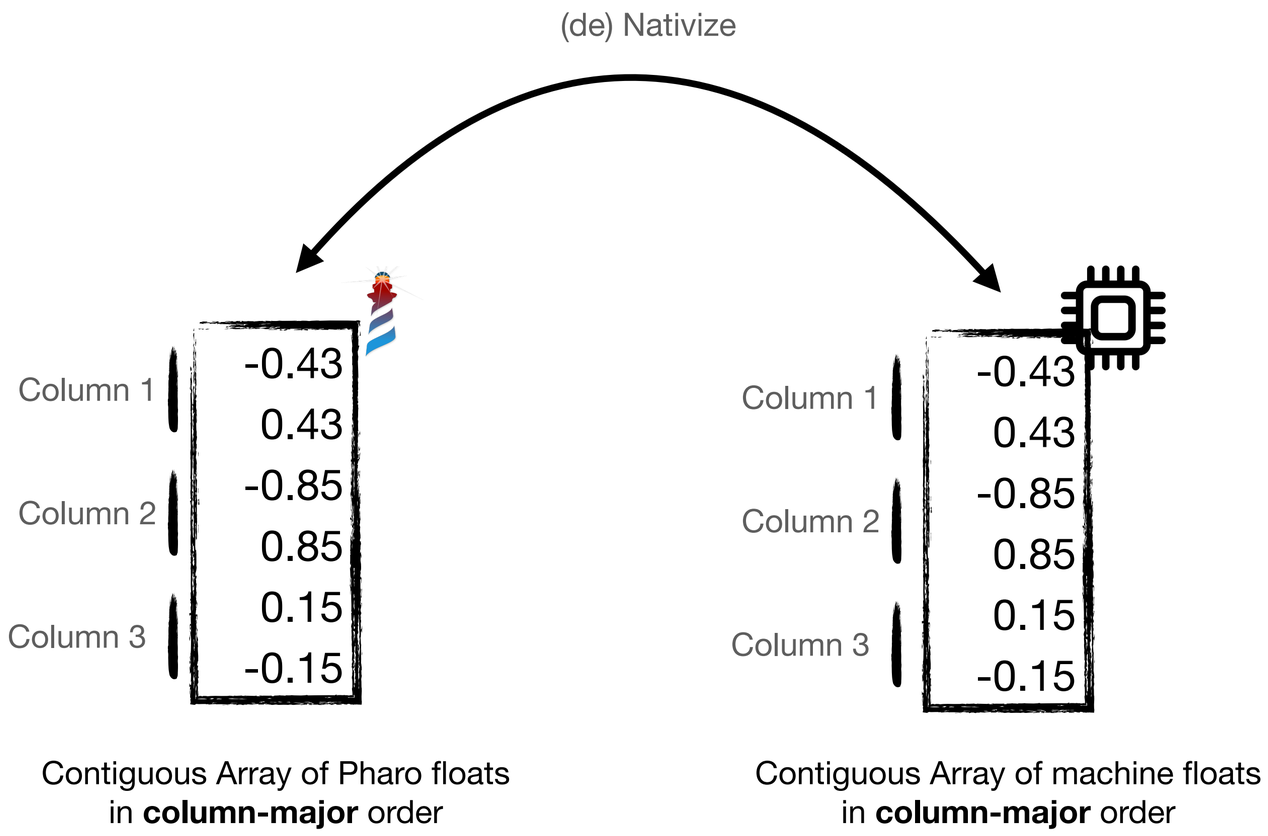
Proposal 1 Performance Profile

- Float boxing unboxing hurts
  - average computation
  - data movement inside Pharo
- Easily optimisable
  - Vectorisation
  - Or generic primitives



Proposal 2: Contiguous Array of Pharo Floats

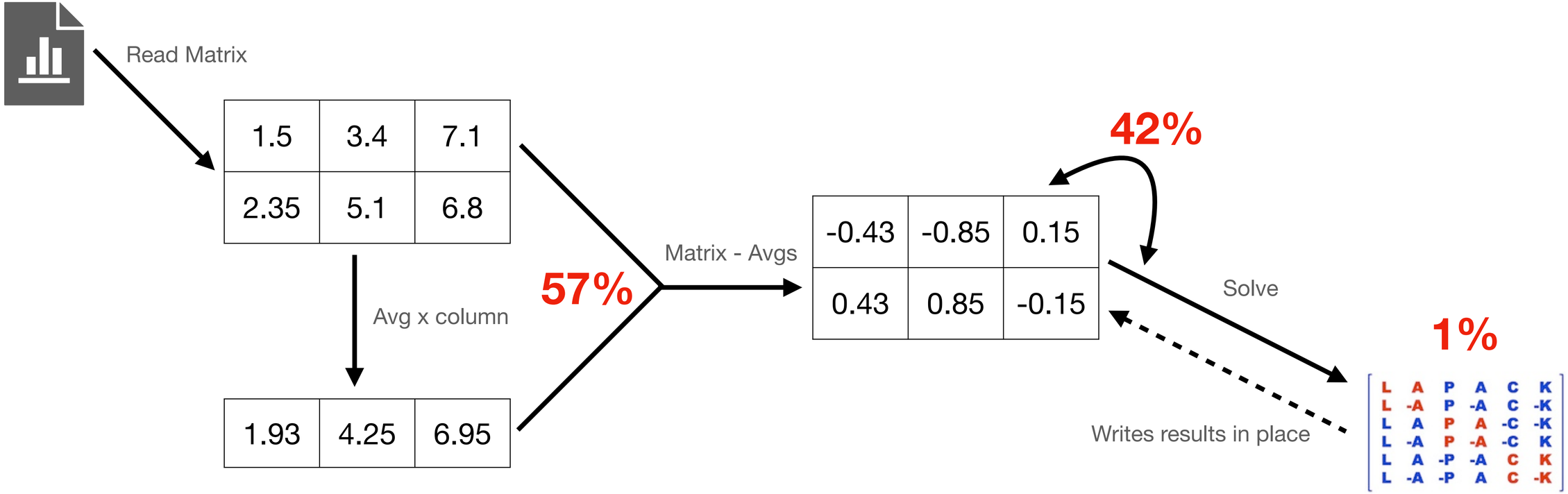
-0.43	-0.85	0.15
0.43	0.85	-0.15



Pharo-AI representation

Lapack's expectations

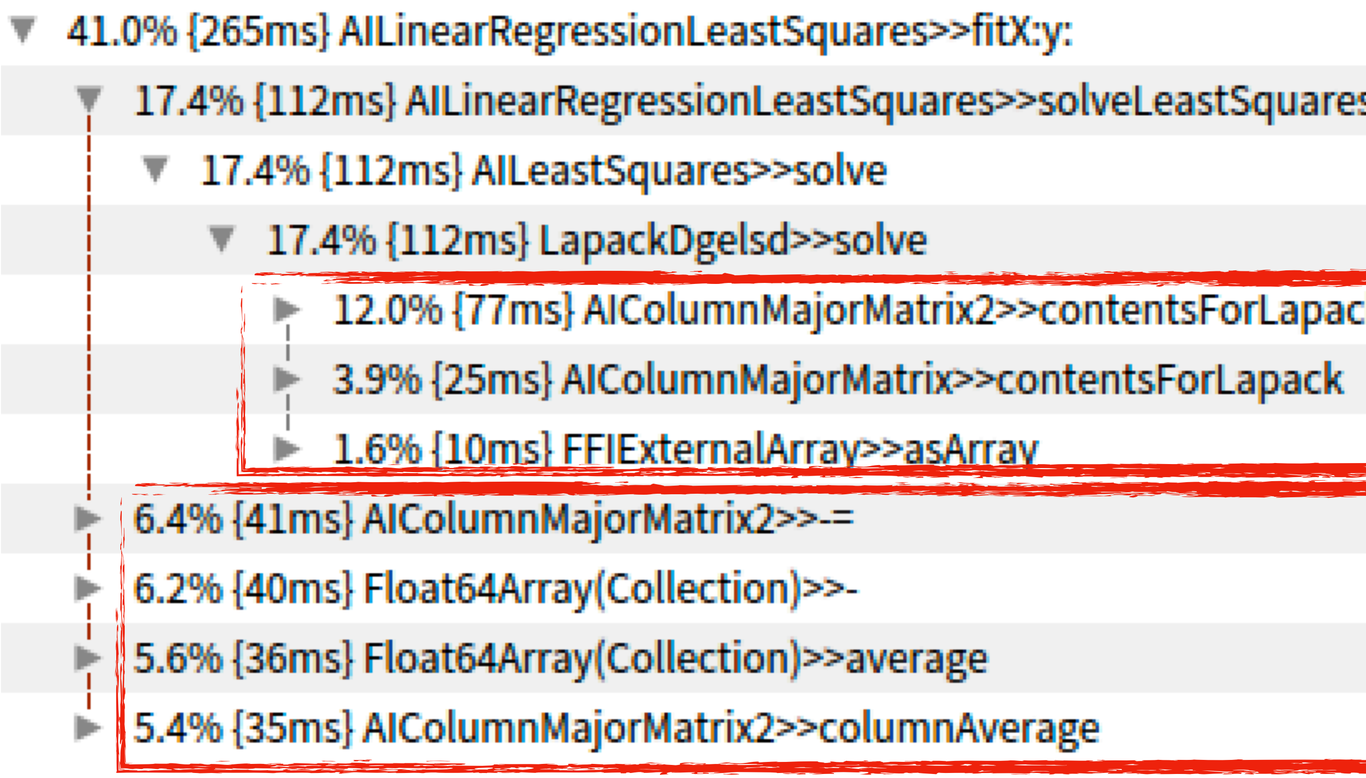
Proposal 2 Performance



But 5.6x faster than baseline!

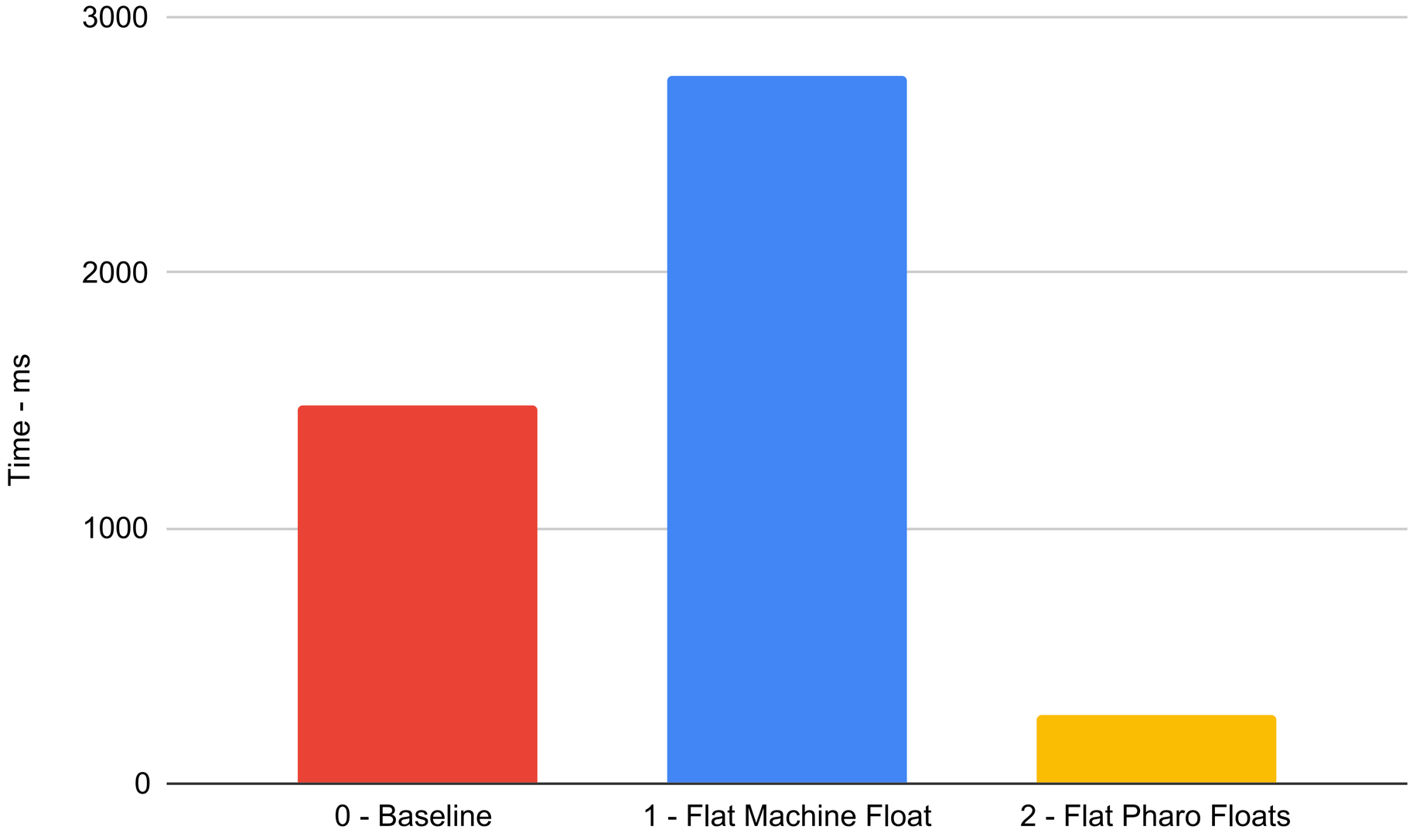
Proposal 2 Performance Profile

- Large data operations hurt (less)
  - e.g., averages…
  - Could be further optimised



- Transforming to native floats
  - Hurts! And is not much further optimisable
  - Data movement will bound what we can achieve…

First Result Summary



Conclusion

- Hypothesise, measure and profile before making decisions
- **Good design** has more **performance impact** than a good compiler
- Solutions are bound by different properties
  - language performance => improved with a better implementation
  - data movement => maybe avoidable, improvable with faster machines :)